data analysis with R in a nutshell

prof. dr. nicolas meseth

1. work environment
2. load and inspect
3. transform
4. visualise
5. self-study

before you build,
you set up the
workbench.

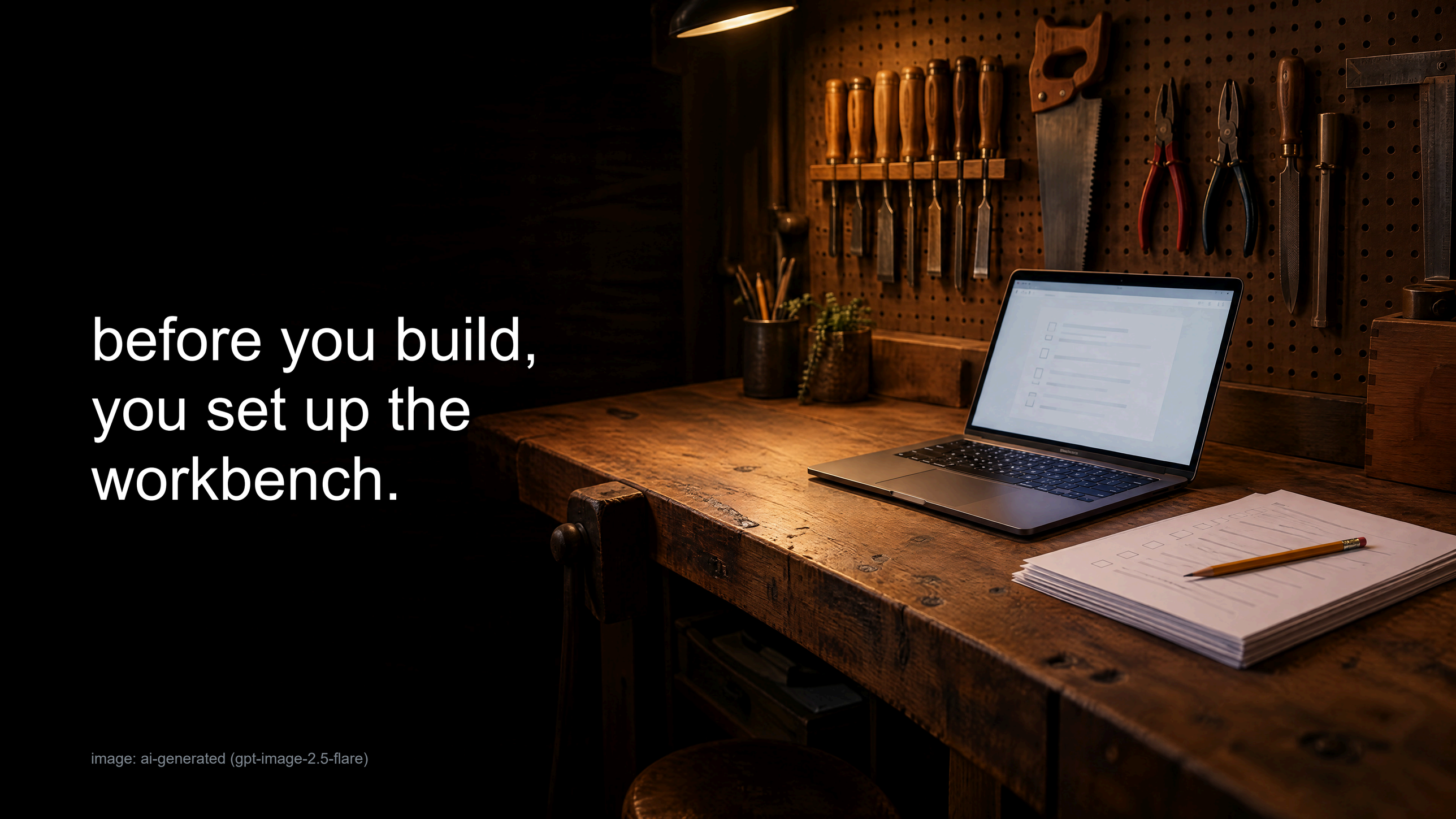


image: ai-generated (gpt-image-2.5-flare)

quick poll

how are your Excel skills?

hardly ever used it

simple formulas

pivot tables and lookups



live.hsos.uber.space/q

quick poll

how are your Excel skills?

hardly ever used it

simple formulas

pivot tables and lookups

live.hsos.uber.space/q

quick poll

how confident do you feel in R?

never used it

a little

quite confident



live.hsos.uber.space/q

quick poll

how confident do you feel in R?

never used it

a little

quite confident



live.hsos.uber.space/q

quick poll

how confident are you in visualising data?

hardly ever made a chart

charts in Excel or PowerPoint

i choose the chart type on purpose

live.hsos.uber.space/q

quick poll

how confident are you in visualising data?

hardly ever made a chart

charts in Excel or PowerPoint

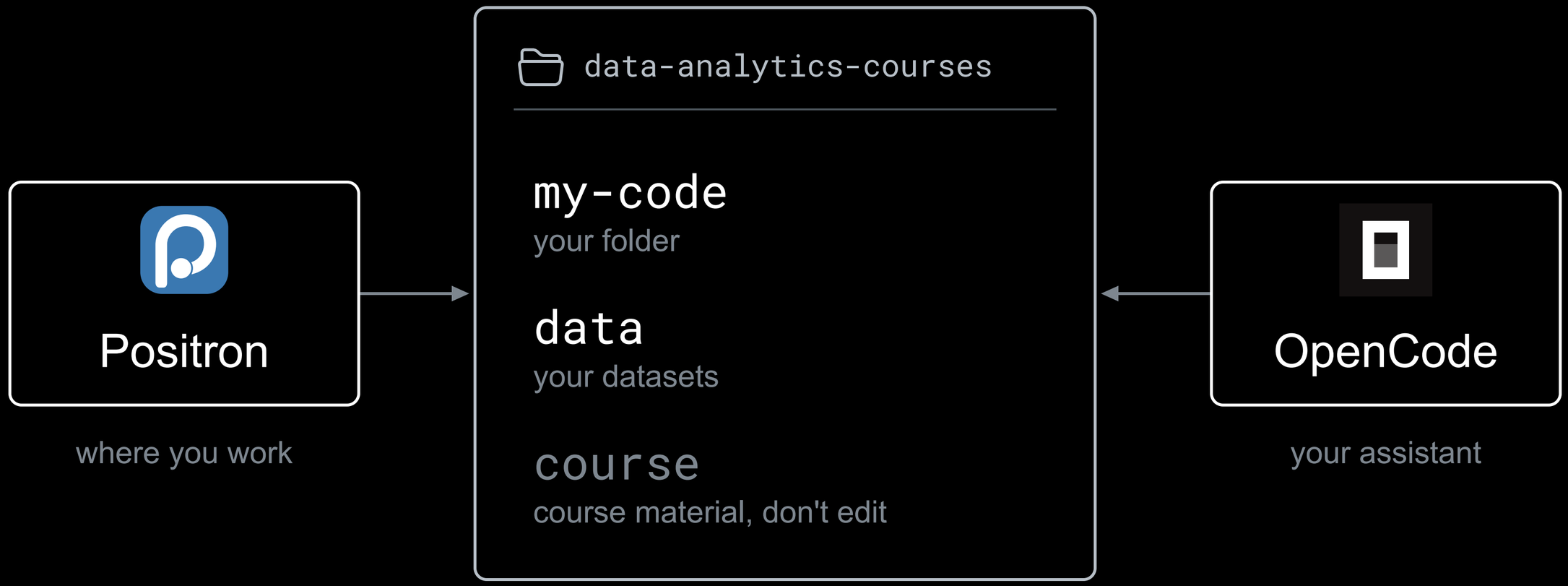
i choose the chart type on purpose

live.hsos.uber.space/q

part 1

work environment

two windows, one folder



install OpenCode Desktop

`opencode.ai/download`

install OpenCode Desktop `opencode.ai/download`

download the course folder `github.com/winf-hsos/data-analytics-courses`

install OpenCode Desktop	`opencode.ai/download`
download the course folder	`github.com/winf-hsos/data-analytics-courses`
unzip*, open in OpenCode	`Add project`

* Windows: right-click, „extract all“. double-clicking the ZIP file does not unzip it.

install OpenCode Desktop	`opencode.ai/download`
download the course folder	`github.com/winf-hsos/data-analytics-courses`
unzip*, open in OpenCode	`Add project`
type in OpenCode	`/onboarding`

* Windows: right-click, „extract all“. double-clicking the ZIP file does not unzip it.

from here on,
your assistant
sets up the rest.



image: ai-generated (gpt-image-2.5-flare)

looks for Positron, R, Quarto

already there? nothing to do

looks for Positron, R, Quarto

already there? nothing to do

installs what is missing

it installs, or it guides you

what /onboarding does

looks for Positron, R, Quarto — already there? nothing to do

installs what is missing — it installs, or it guides you

packages, data, system check — `my-code/systemcheck.txt`

it asks before every installation.

your assistant keeps notes on your work.

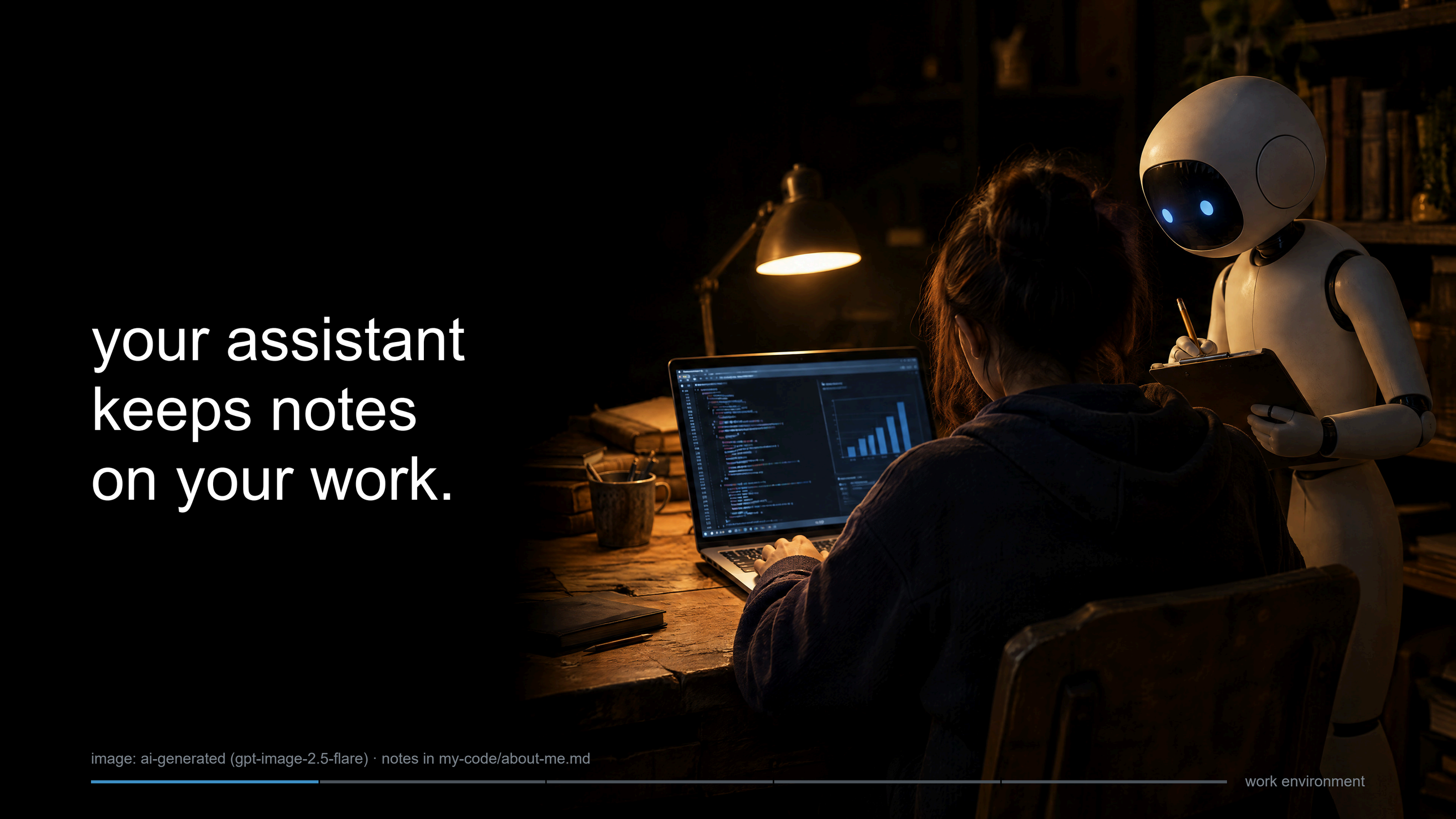


image: ai-generated (gpt-image-2.5-flare) · notes in my-code/about-me.md

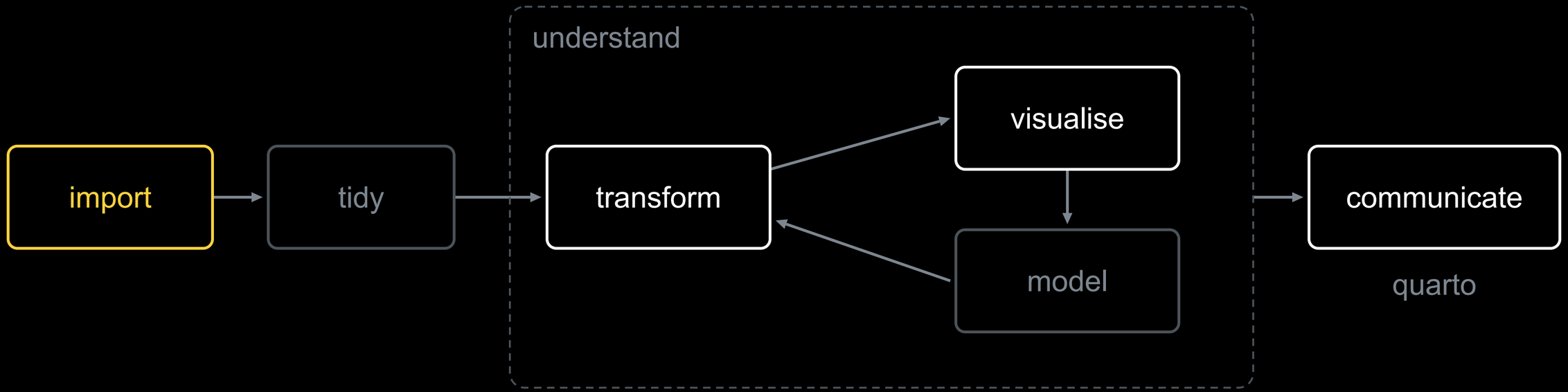
OpenCode	open the course folder › type `/onboarding`
ILIAS	download the dataset › into `data/`
Positron	`File` › `Open Folder` › the course folder

done when Positron shows `my-code`, `data` and `course` on the left.

part 2

load and inspect

exploratory data analysis



after Wickham, Çetinkaya-Rundel, Grolemund: R for Data Science, 2nd ed.

the data: a CSV file

the file: plain text

```
q002geburt,q003land,D043kinderzahl,D044stadtf
1970,1,NA,Kleinstadt <5000
1990,7,0,Großstadt
1963,6,NA,Großstadt
```

from a German Excel with `;` and decimal commas: `read_csv2()`

the data: a CSV file

the file: plain text

```
q002geburt,q003land,D043kinderzahl,D044stadtf
1970,1,NA,Kleinstadt <5000
1990,7,0,Großstadt
1963,6,NA,Großstadt
```

first line: the column names

one line = one respondent

`,` next column `NA` missing

from a German Excel with `;` and decimal commas: `read_csv2()`

the data: a CSV file

the file: plain text

```
q002geburt,q003land,D043kinderzahl,D044stadtf
1970,1,NA,Kleinstadt <5000
1990,7,0,Großstadt
1963,6,NA,Großstadt
```

first line: the column names

one line = one respondent

`read_csv()`

`,` next column `NA` missing

q002geburt	q003land	D043kinderzahl	D044stadtf
1970	1	NA	Kleinstadt <5000
1990	7	0	Großstadt
1963	6	NA	Großstadt

a tibble:
3 rows, 4 columns

from a German Excel with `;` and decimal commas: `read_csv2()`

```
library(tidyverse)
survey <- read_csv("data/mds12_schoko_milch.csv")
```

the result gets the name `survey` and appears in the variables pane on the right.

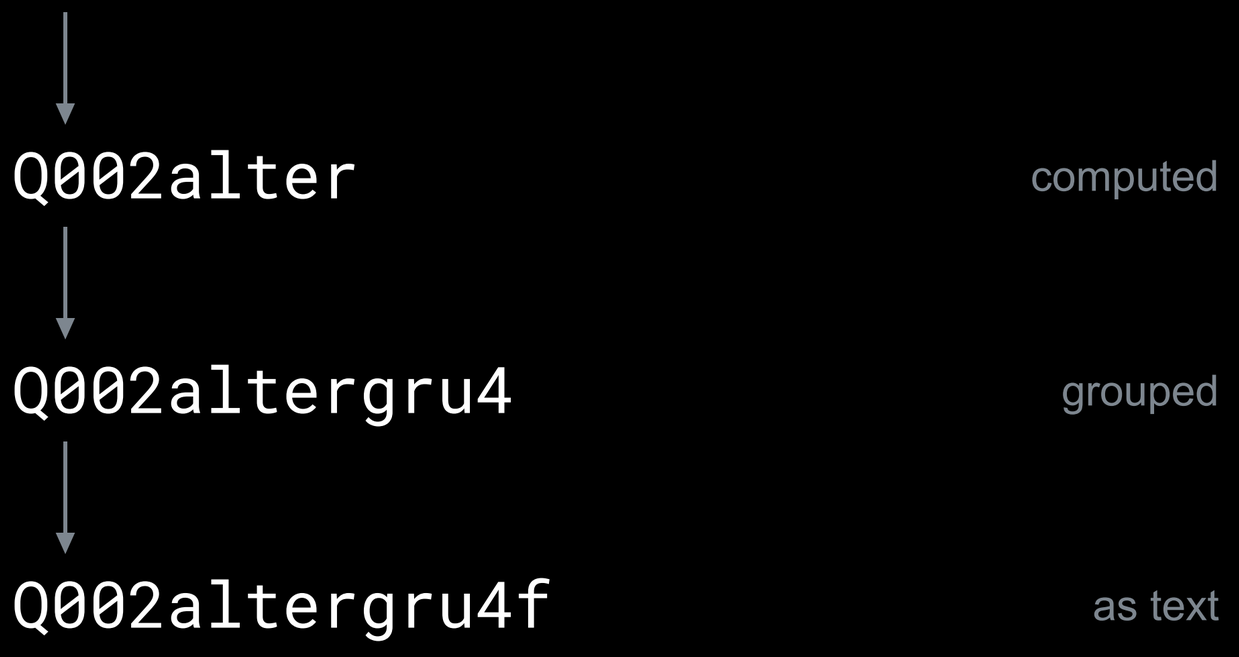
tip for other data: `janitor::clean_names()` tidies messy column names. not here: it would lowercase `Q002alter` and break the raw/prepared system.

```
survey |>
  glimpse()
```

2,811 rows, one per respondent. 606 columns, one per question.

also useful: `nrow()` · `ncol()` · `names()` · `summary()` · `skimr::skim()`

q	quota, screening
v	behaviour
p	psychographic scales
d	demographics
m	milk
s	chocolate



lowercase = raw, uppercase = prepared, f = as text

the pipe: the result becomes the first argument

```
survey |> count(      , D044stadtf)
```



the result on the left becomes the first argument on the right

the pipe: the result becomes the first argument

```
survey |> count(      , D044stadtf)
```



the result on the left becomes the first argument on the right

is the same as `count(survey, D044stadtf)`

without and with the pipe

without: read from the inside out

```
arrange(count(filter(survey, Q002alter >= 60), D044stadtf), desc(n))
```

without and with the pipe

without: read from the inside out

```
arrange(count(filter(survey, Q002alter >= 60), D044stadtf), desc(n))
```

with the pipe: read from top to bottom, „and then“

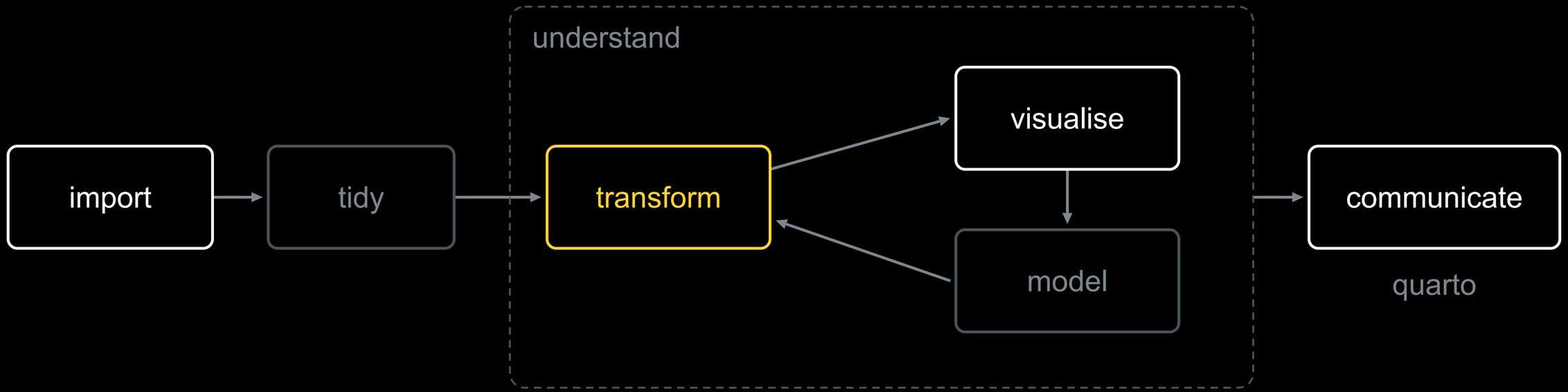
```
survey |>
  filter(Q002alter >= 60) |>
  count(D044stadtf) |>
  arrange(desc(n))
```

type `|>` in Positron: `Ctrl+Shift+M` (Mac: `Cmd+Shift+M`)

part 3

transform

exploratory data analysis



```
survey |>
  filter(Q002alter >= 60) |>
  count(D044stadtf)
```

`filter()` keeps rows

```
survey |>
  select(Q002alter, D044stadtf) |>
  glimpse()
```

`select()` keeps columns

```
survey |>
  mutate(is_60_plus = Q002alter >= 60) |>
  count(is_60_plus)
```

yes/no columns start with `is_`.

```
survey |>
  group_by(D044stadtf) |>
  summarise(mean_age = mean(Q002alter),
            n_respondents = n()) |>
  arrange(mean_age)
```

mean_ is a mean, n_ a count.

who does this number speak about? look at n.

tip: tidylog tells you what each step did

```
library(tidylog)

survey |>
  filter(Q002alter >= 60) |>
  count(D044stadtf)


filter: removed 2,042 rows (73%), 769 rows remaining
count: now 7 rows and 2 columns, ungrouped
```

load it after the tidyverse. every `filter()`, `mutate()`, `summarise()` then reports its effect in the console.

try it: dplyr verbs

`select` `filter` `arrange` `mutate` `group_by + summarise` `summarise or mutate?`

```
survey |>
  filter(age >= 50)
```

before: six respondents: age in years, size of place, persons in the household

id	age	city	household
~~1~~	~~23~~	~~city~~	~~1~~
2	67	village	2
~~3~~	~~45~~	~~town~~	~~4~~
~~4~~	~~34~~	~~city~~	~~3~~
5	58	village	2
6	71	town	1

after: 3 rows, 4 columns

id	age	city	household
2	67	village	2
5	58	village	2
6	71	town	1

numbers without meaning

```
survey |>
  count(q0031and)
```

state 1 to 16. which is which?
that is not in the data but in the codebook, p. 7.

```
# codes from the study documentation, p. 7
state_names <- c(
  "Baden-Württemberg", "Bavaria", "Berlin", "Brandenburg",
  "Bremen", "Hamburg", "Hesse", "Mecklenburg-Western Pomerania",
  "Lower Saxony", "North Rhine-Westphalia", "Rhineland-Palatinate",
  "Saarland", "Saxony", "Saxony-Anhalt", "Schleswig-Holstein",
  "Thuringia"
)

survey <- survey |>
  mutate(state = factor(q003land, levels = 1:16, labels = state_names))

survey |>
  count(state)
```

the quotas from the codebook

```
# quotas from the study documentation, p. 7
quotas <- tibble(
  state = state_names,
  pct_quota = c(
    12.43, 14.38, 4.86, 3.75, 1.90, 3.05, 7.47, 2.78,
    9.21, 19.22, 5.33, 2.21, 5.25, 3.34, 4.15, 3.31
  )
)
```

which share of respondents should come from each state.

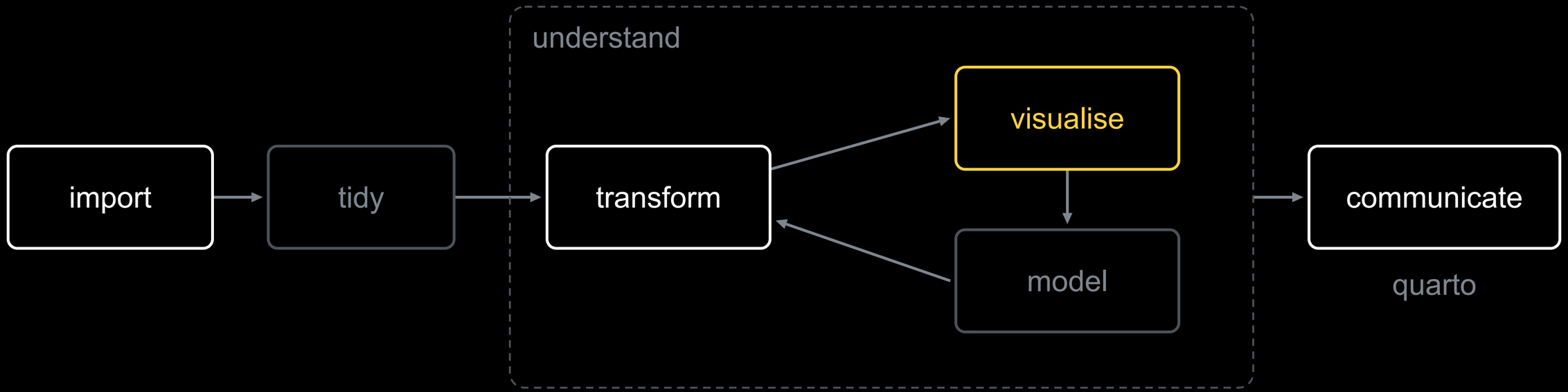
```
respondents_by_state <- survey |>
  count(state, name = "n_respondents") |>
  left_join(quotas, by = "state") |>
  mutate(
    pct_sample = 100 * n_respondents / sum(n_respondents),
    diff_quota_pp = pct_sample - pct_quota
  )
```

`left_join()` adds the matching row of the other table to every row.

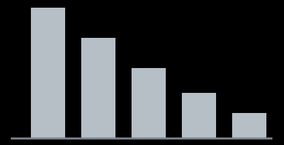
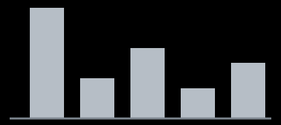
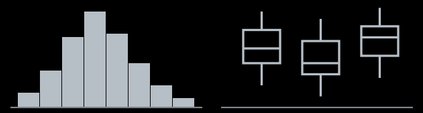
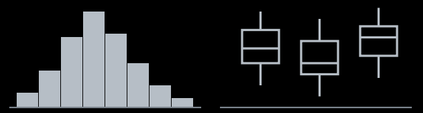
part 4

visualise

exploratory data analysis



which plot? it depends on the scale

	categorical		metric	
	nominal	**ordinal**	**interval**	**ratio**
example	state Bavaria, Hesse, …	size of place big city … village	year of birth 1958, 1987, …	age 23, 67, …
in R	`<fct>`	`<fct> ordered`	`<dbl>`	`<dbl>`
summary	`count()`	`count(), median()`	`mean(), sd()`	`mean(), sd()`
plot				
	bar chart, by frequency	bar chart, in scale order	histogram, boxplot	histogram, boxplot

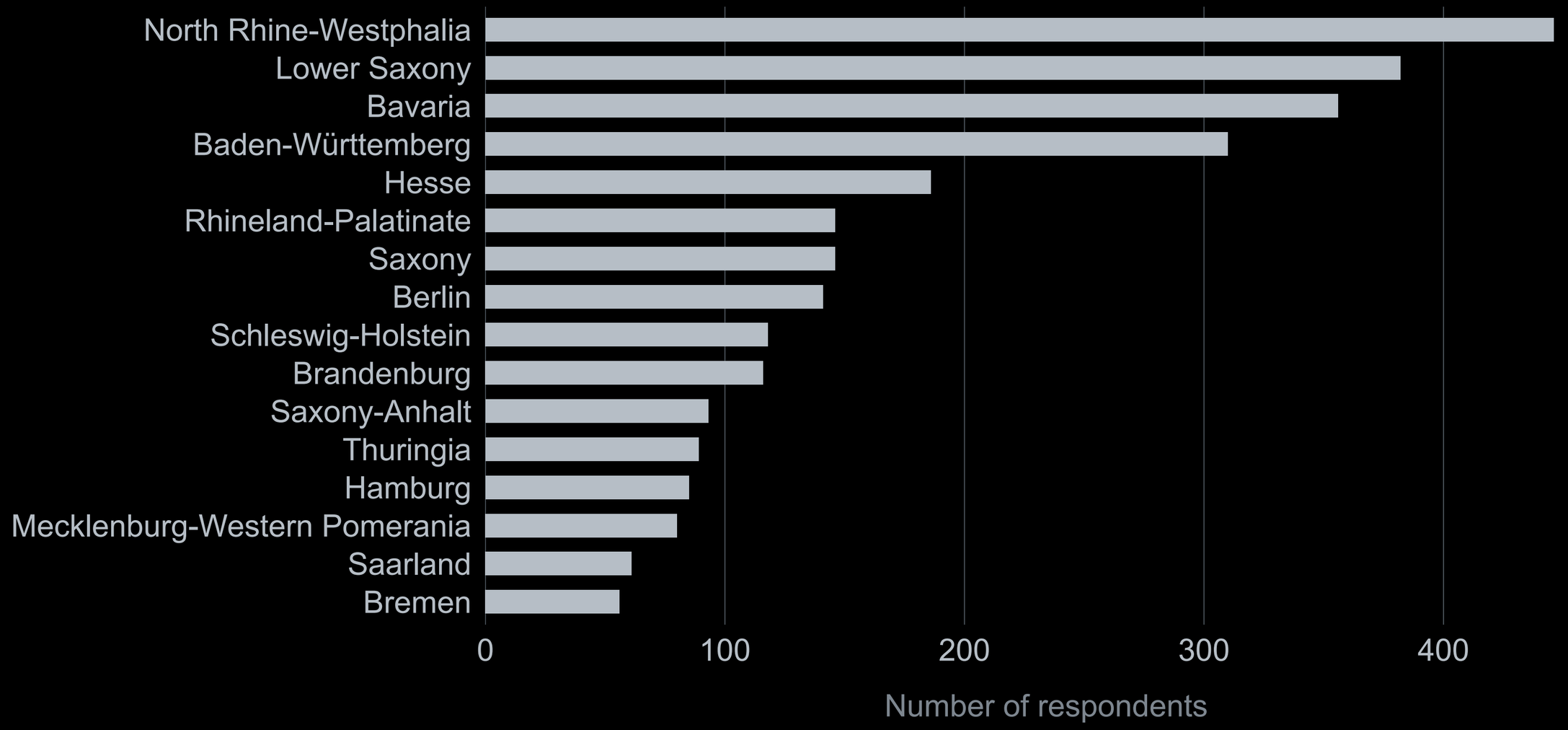
the scale decides the data type, the summary and the plot.

nominal: a bar chart, by frequency

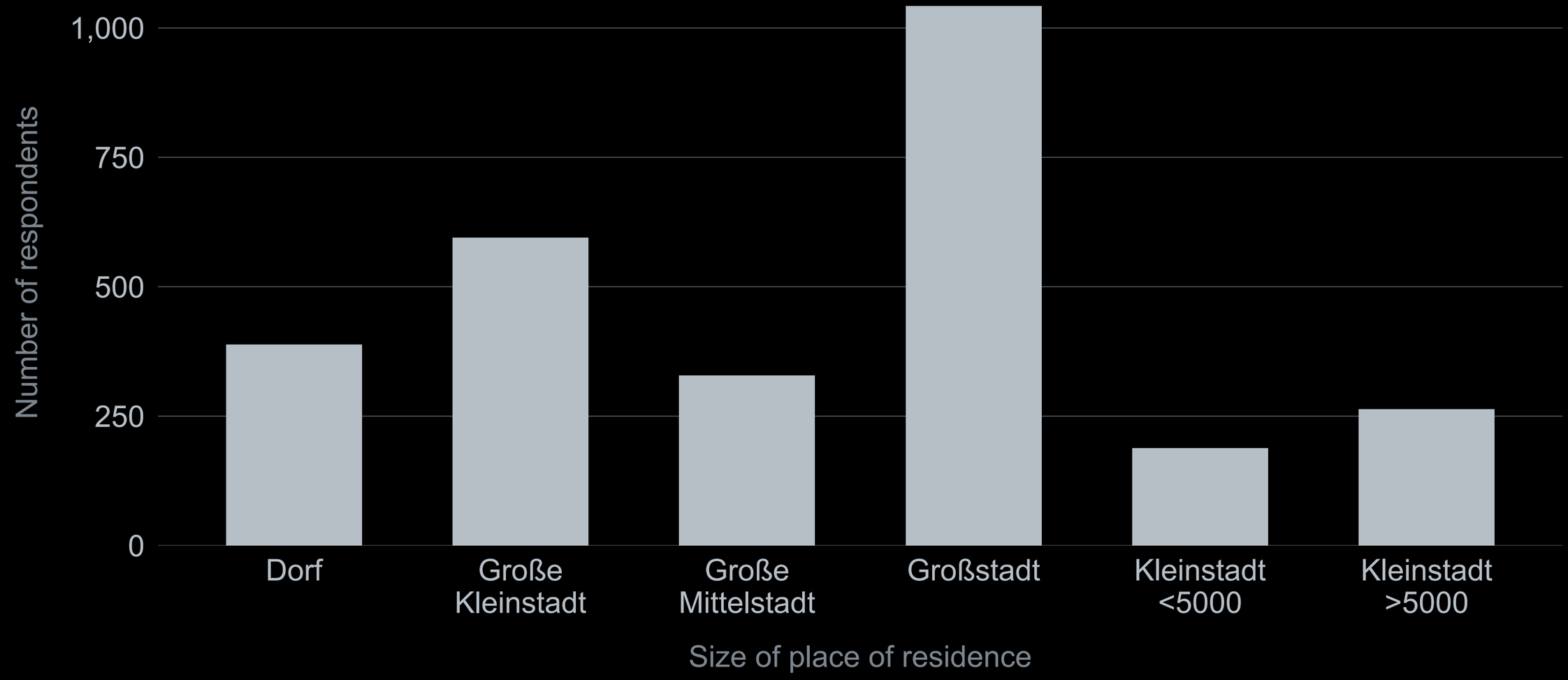
```
survey |>
  ggplot(aes(y = fct_rev(fct_infreq(state)))) +
  geom_bar() +
  labs(x = "Number of respondents", y = NULL)
```

inside `ggplot()`, `+` adds layers, not `|>`.

respondents by state



as text: alphabetical



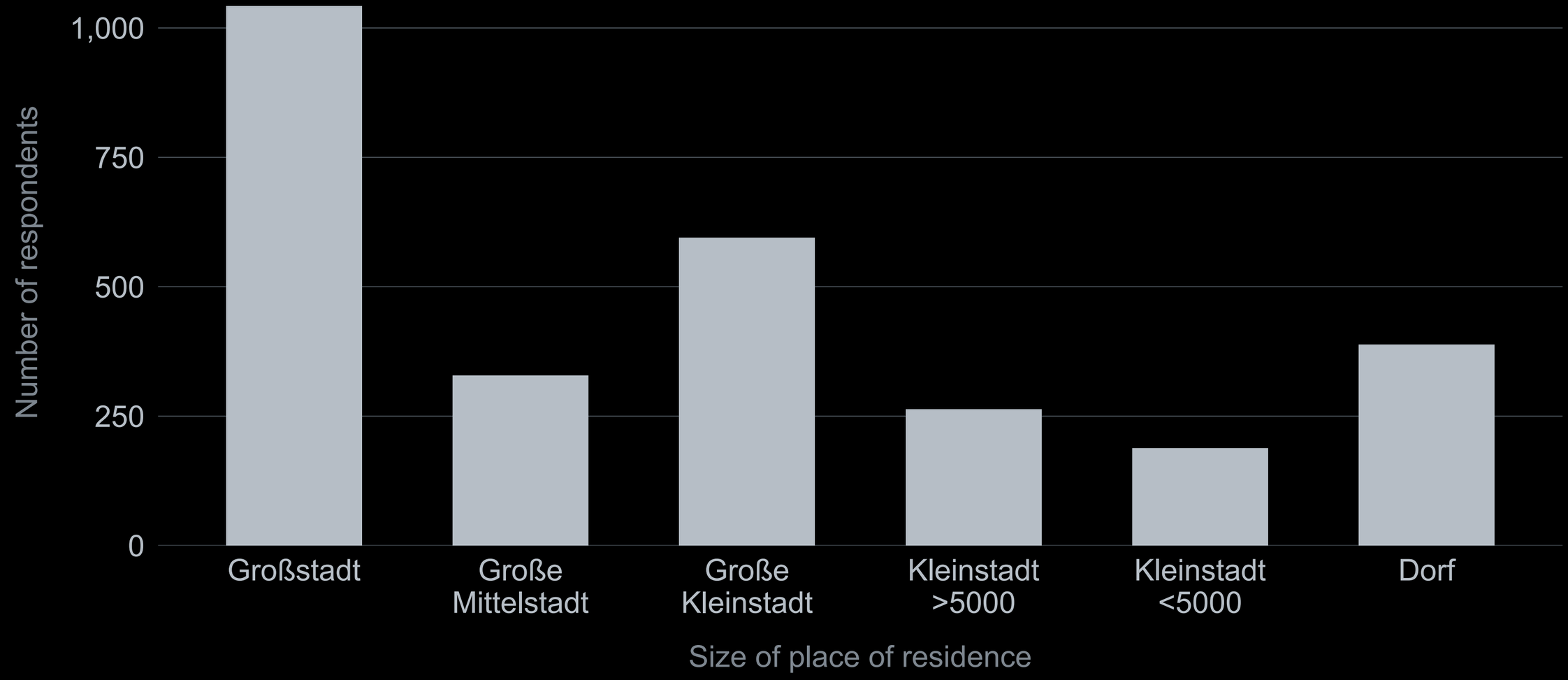
ordinal: a factor keeps the order of the scale

```
survey <- survey |>
  mutate(city_size = factor(D044stadtf, levels = c(
    "Großstadt", "Große Mittelstadt", "Große Kleinstadt",
    "Kleinstadt >5000", "Kleinstadt <5000", "Dorf"
  )))

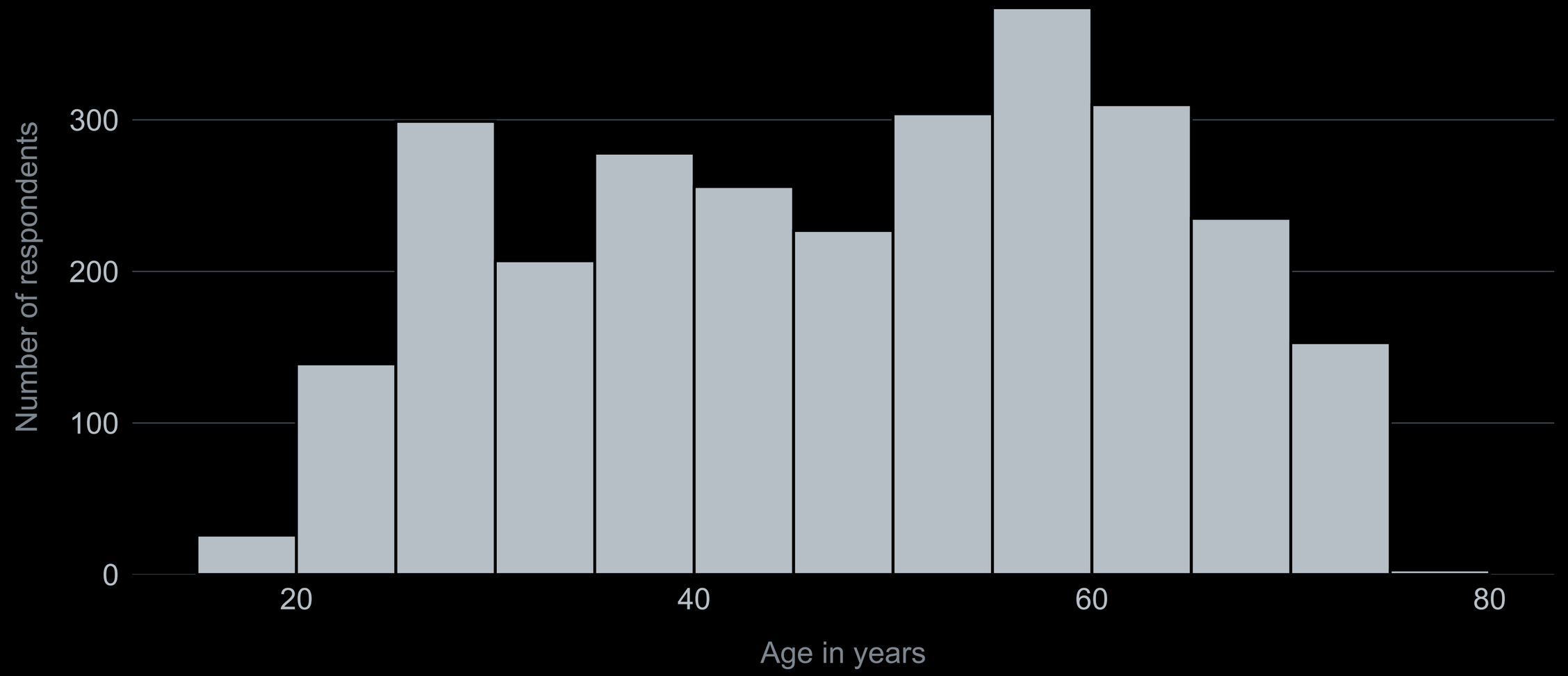
survey |>
  filter(!is.na(city_size)) |>
  ggplot(aes(x = city_size)) +
  geom_bar() +
  labs(x = "Size of place of residence", y = "Number of respondents")
```

text sorts alphabetically. a factor keeps the order you give it.

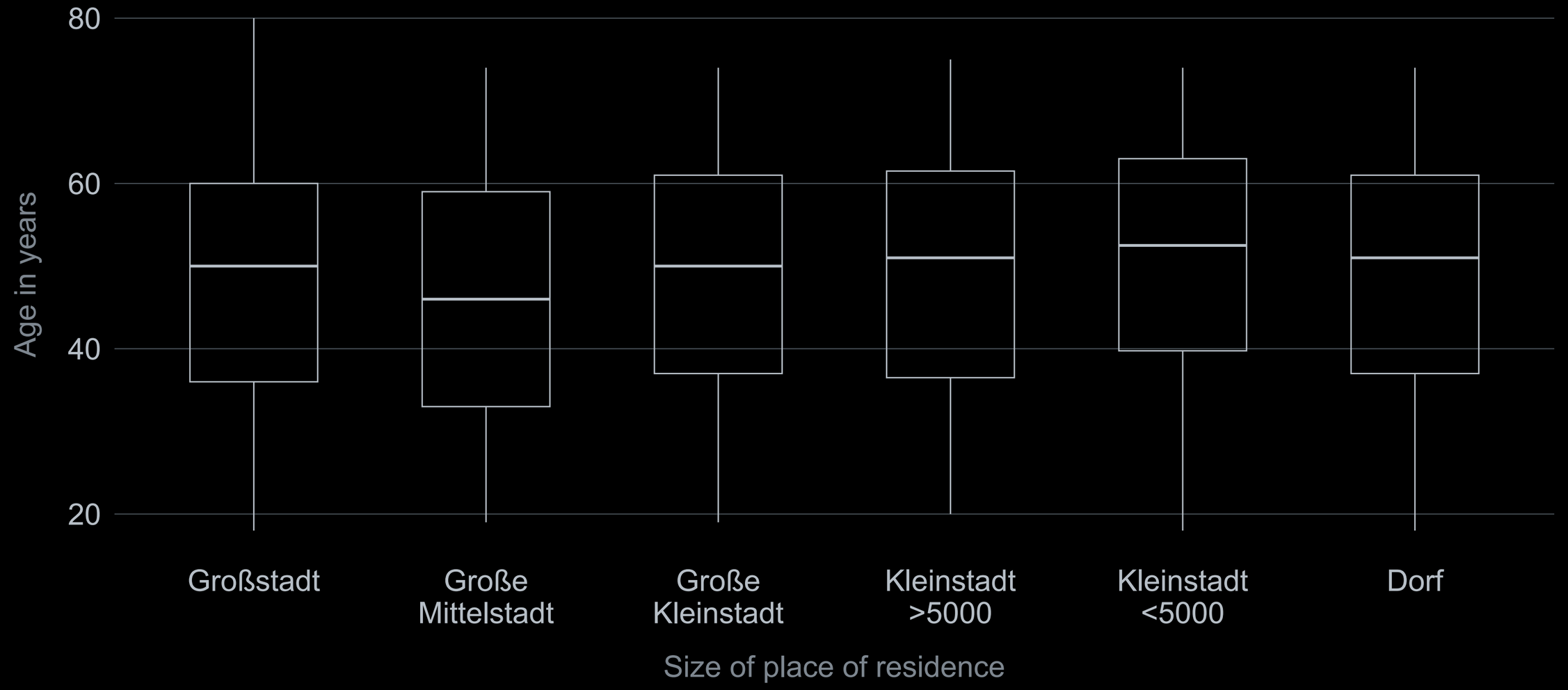
as factor: in the order of the scale



one number: a histogram

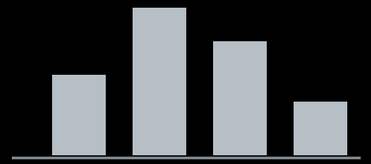


a number by group: a boxplot



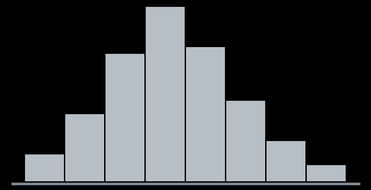
which plot for which question?

bar chart



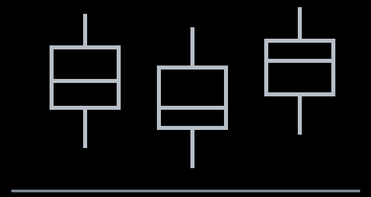
how many
per category?

histogram



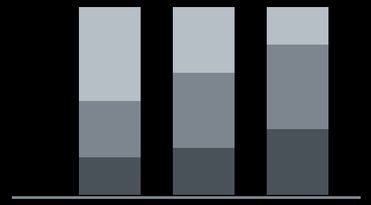
how is one number
spread?

boxplot



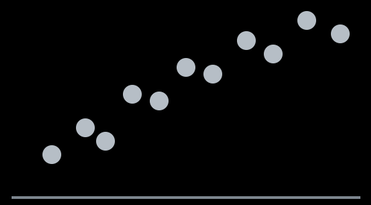
does a number differ
between groups?

stacked bars, 100 %



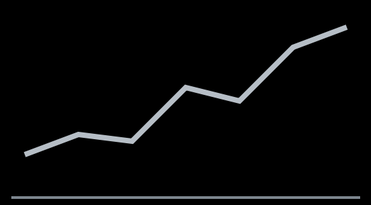
does a category differ
between groups?

scatter plot



do two numbers
go together?

line chart



how does it change
over time?

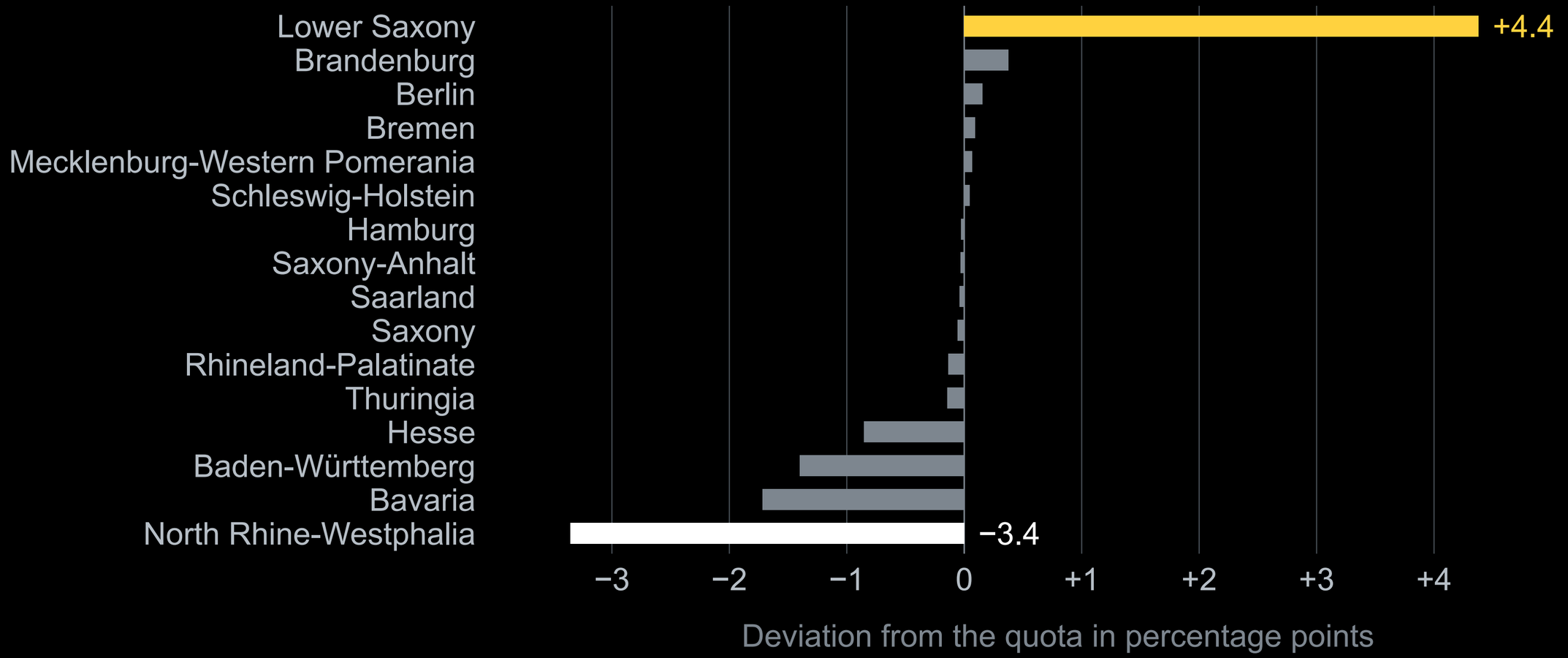
more in Claus Wilke: Fundamentals of Data Visualization, directory of visualizations

remember the join

```
respondents_by_state <- survey |>
  count(state, name = "n_respondents") |>
  left_join(quotas, by = "state") |>
  mutate(
    pct_sample = 100 * n_respondents / sum(n_respondents),
    diff_quota_pp = pct_sample - pct_quota
  )
```

one row per state. the next figure shows `diff_quota_pp`.

sample against quota

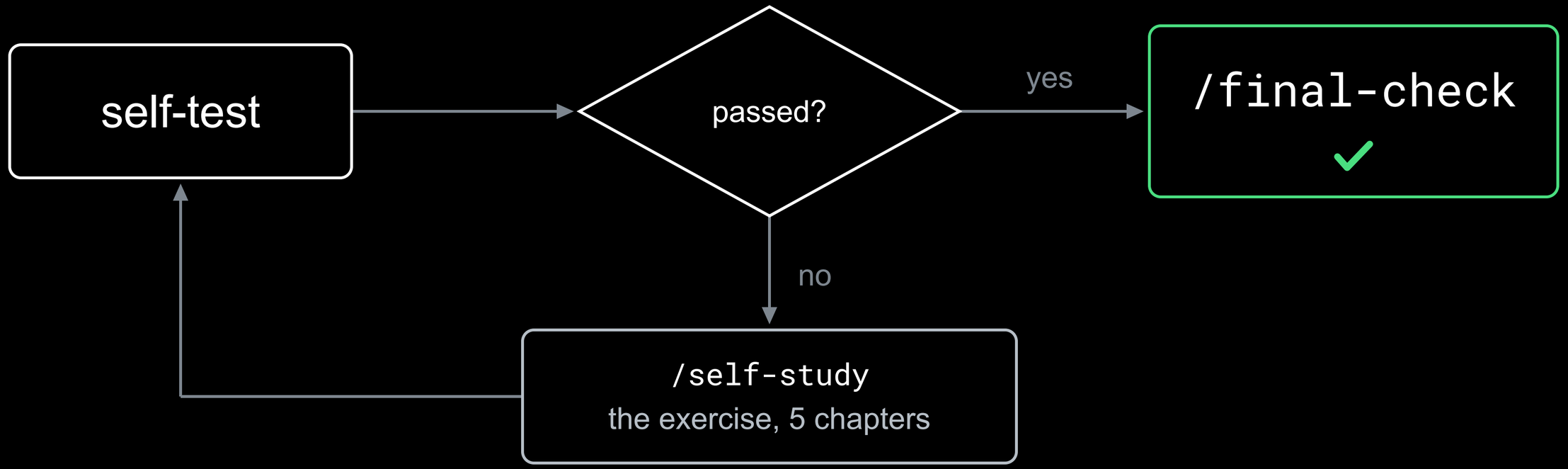


how the data came about is not in the data.

part 5

self-study

test first, then learn



one exercise, five chapters

where do respondents shop, and who buys organic?

1	get to know the data	1.5 h
2	describe single variables	3 h
3	compare groups	3 h
4	figures	2–3 h
5	answer one question on your own	2 h

in `my-code/self-study/`, every task with its reading from R for Data Science.

```
answer_6 <- NULL   # replace NULL with your code

check_answer(6, answer_6)
```

`check_answer()` tells you right away whether your value is correct,
and if not, where to look.

your assistant could solve this test for you.
it cannot pass it for you.

`/self-study`

chapter by chapter: reading, hints, a check

two commands

`/self-study`	chapter by chapter: reading, hints, a check
`/final-check`	runs your self-test, then asks about your code

no grade. it shows you whether you can explain your numbers.

does

explains as often as you like
reads error messages
writes code in small steps
keeps to the course standards

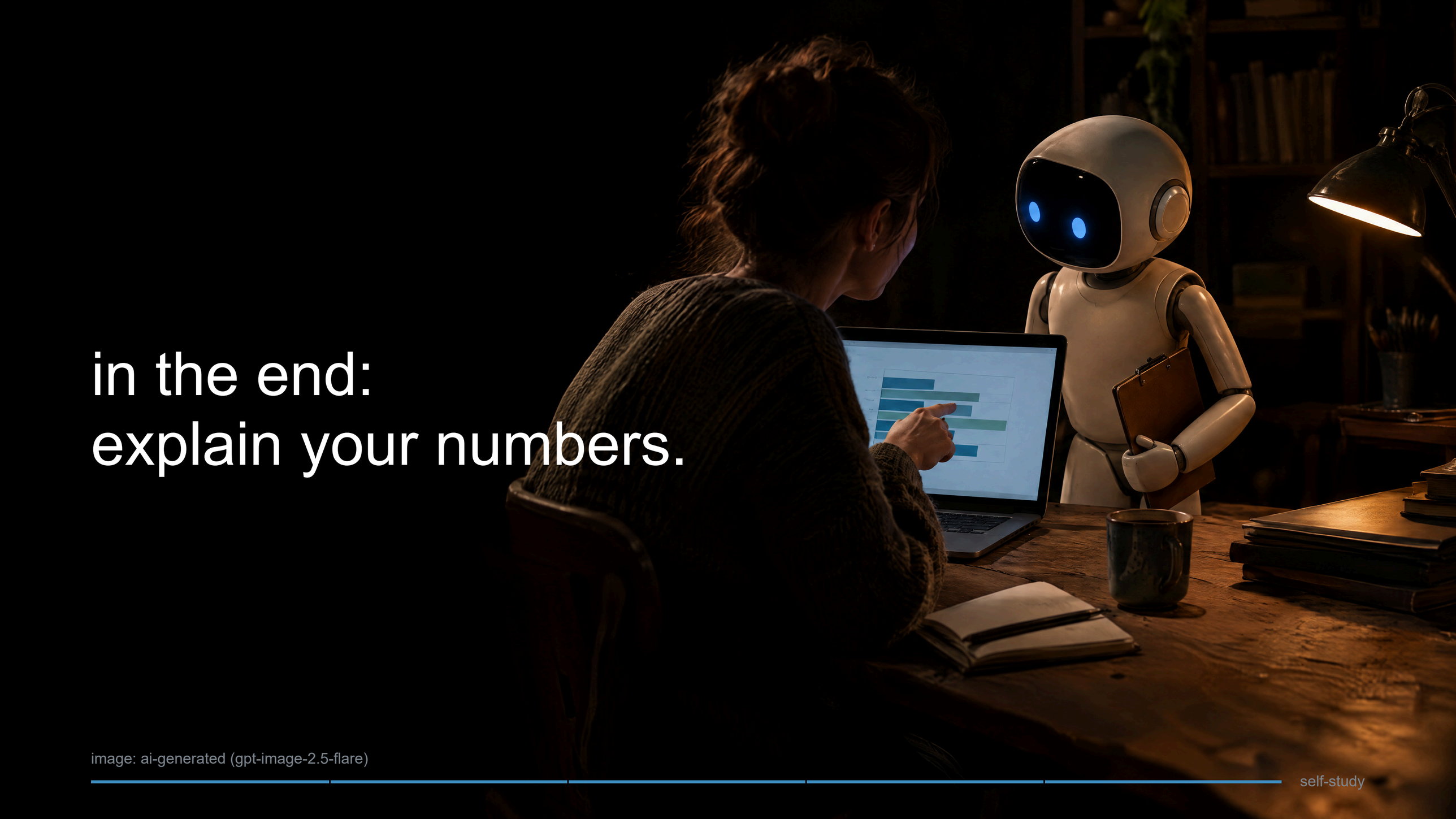


doesn't

solve the self-test
decide what you investigate
take responsibility for the result



the agent can write code. you decide the what and own the result.



in the end:
explain your numbers.

image: ai-generated (gpt-image-2.5-flare)